



北京師範大學
BEIJING NORMAL UNIVERSITY

《计算机图形学》课程实验

实验五：模型加载与材质 (参考操作教程)

授课教师：张鸿文

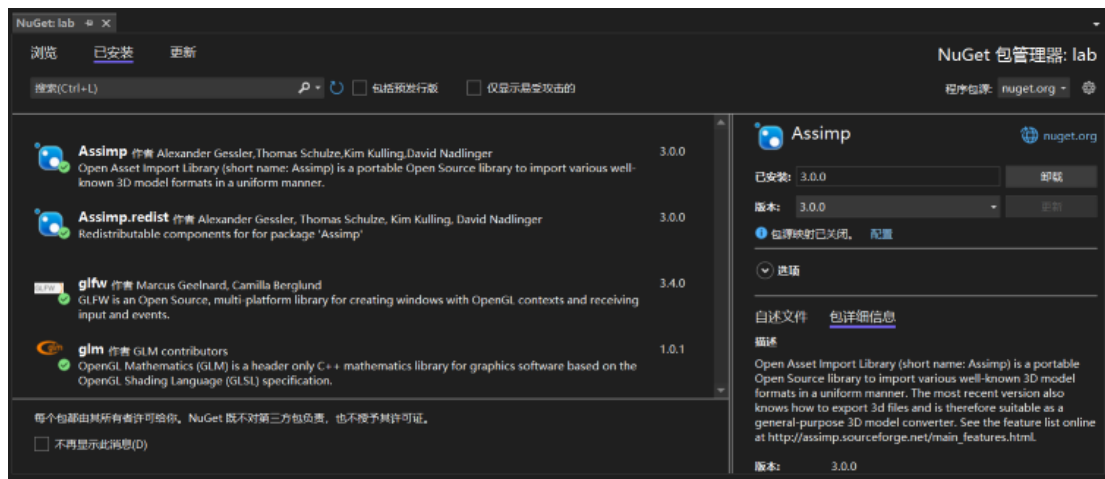
助教：李玉慧

课程网页：<https://zhanghongwen.cn/cg>

计算机图形学实验五——模型加载与材质

1. 安装 ASSIMP 库

直接从 NuGet 程序包管理器安装，最新稳定版为 3.0.0 版



2. 复制并运行示例代码“model_loading.cpp”

https://learnopengl.com/code_viewer_gh.php?code=src/3.model_loading/1.model_loading/model_loading.cpp

从代码网页中单击带下划线的文本，复制所需头文件，新建到项目中。在之前示例代码头文件的基础上，需要在项目中添加“mesh.h”与“model.h”文件。

```
#include <learnopengl/shader_m.h>
#include <learnopengl/camera.h>
#include <learnopengl/model.h>
```

```
Shader ourShader("1.model_loading.vs", "1.model_loading.fs");
```

<https://learnopengl-cn.github.io/data/backpack.zip>

从教程链接中下载修改后的模型文件，并解压到项目文件夹中。后续模型文件会根据你在设置的路径读取，注意分辨“解决方案”与“文件夹”的组织架构的区别。

“model.h”中引用了“stb_image.h”，如果你已经在项目中新建“stb_image_swap.cpp”文件，即可直接运行；如果你采用在之前的主代码中直接添加“#define STB_IMAGE_IMPLEMENTATION”的方式，也需要在“model.h”中添加。

```
#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"
```

修改示例代码中的#include 部分与文件路径（头文件内也会引用自建头文件，注意复制并修改）。具体路径需根据你的路径修改。

“model_loading.cpp”

```
#include "shader_m.h"
#include "camera.h"
```

```
#include "model.h"

.....

Shader ourShader("1.model_loading.vs", "1.model_loading.fs");

.....

Model ourModel("./backpack/backpack.obj");
```

“model.h”

```
#include "mesh.h"
#include "shader.h"
```

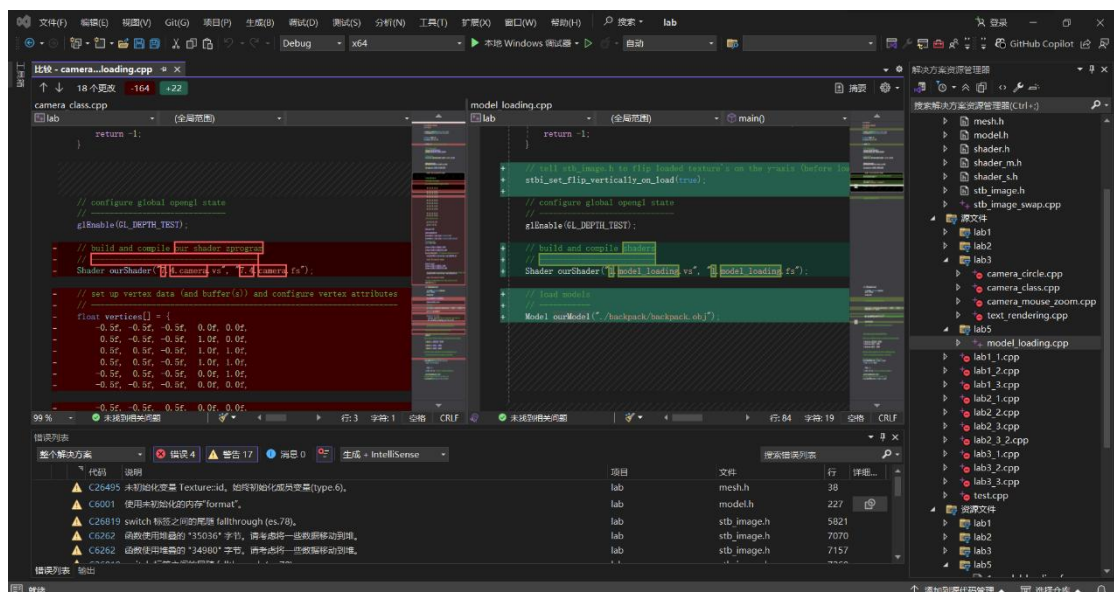
“mesh.h”

```
#include "shader.h"
```

修改后直接运行，可通过键鼠操作观察；由于鼠标被限制使用，可以通过快捷键截图。

3. 对比“model_loading.cpp”与“camera_class.cpp”

对比代码可知，前者将模型（包括形状、纹理等）的读取集成到 Model 类中，在示例代码中实例化为 OurModel 变量，以简化主函数。

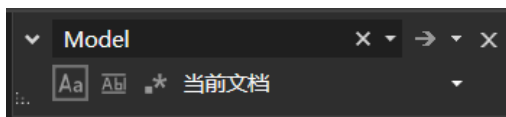


4. 阅读教程“模型加载”一节中的“网格”与“模型”两小节，结合“mesh.h”与“model.h”理解相关概念。

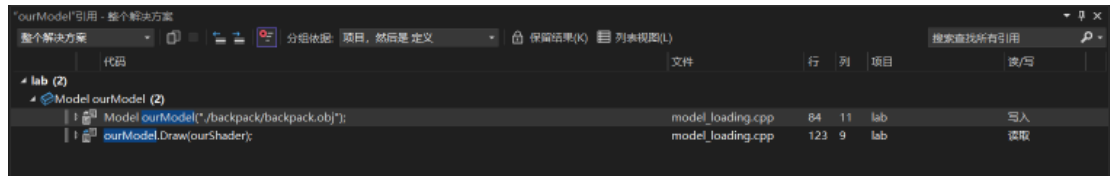
<https://learnopengl-cn.github.io/03%20Model%20Loading/02%20Mesh/>
<https://learnopengl-cn.github.io/03%20Model%20Loading/03%20Model/>

我们根据 Model 类的函数调用，反推各函数的作用，并结合教程分析。

首先在“model_loading.cpp”，使用“Ctrl+F”快捷键，查找“Model”（可在搜索栏选择“区分大小写”）。可以看到该类仅实例化为 ourModel 变量，读入的模型为我们解压的“backpack.obj”。



除了通用查找的方法外，在 VS 中也可以将在代码中用鼠标右击类名（如“Model”）/变量名（如“OurModel”）或函数名（如“Draw”），选择“转到所有引用”，快速查看。



```
Model ourModel("../backpack/backpack.obj");
```

接着查找“ourModel”，除了之前的创建实例外，ourModel 只在渲染循环中调用 Draw 函数执行绘制。

```
ourModel.Draw(ourShader);
```

我们可以直接在“model.h”中查找初始化函数 Model()、绘制函数 Draw()，也可以在“model_loading.cpp”中函数被调用的位置，鼠标右击函数名（如 Draw），选择“速览定义”或“转到定义”，快速查看。

通过对函数的调用分析，在 OurModel 的初始化过程中，共调用了“loadModel”、“processNode”、“processMesh”、“loadMaterialTextures”四个函数，需要结合“模型”与“网格”两节的教程理解代码的含义。

在 loadModel 的核心代码中，实例化 Assimp 的 Importer 类来读取模型文件，设置的选项为“导入三角面片+计算顶点法线+翻转 UV 的 y 轴坐标+计算顶点切线”

```
void loadModel(string const& path)
{
    Assimp::Importer importer;
    const aiScene* scene = importer.ReadFile(path, aiProcess_Triangulate |
aiProcess_GenSmoothNormals | aiProcess_FlipUVs | aiProcess_CalcTangentSpace);
    directory = path.substr(0, path.find_last_of('/'));
    processNode(scene->mRootNode, scene);
}
```

“processNode”、“processMesh”函数的主要作用是，用 Assimp 读取到的层次化模型信息，更新 MyModel 内部的 meshes 变量，其中 mesh 中一定包含每个顶点的坐标(x、y、z)、每个三角面包含的三个顶点指针，可能包含每个顶点的法线方向(nx、ny、nz)、纹理坐标(Tx、Ty)、切线方向(tx、ty、tz)、副切线方向(bx、by、bz)。

```
void processNode(aiNode* node, const aiScene* scene)
{
    .....

    aiMesh* mesh = scene->mMeshes[node->mMeshes[i]];
    meshes.push_back(processMesh(mesh, scene));
    .....
}
```

“TextureFromFile”、“loadMaterialTextures”函数的主要作用是，用 Assimp 读取到的纹理图像、法线贴图、材质信息（漫反射、镜面反射、环境光照、高度图）等，最终将得到的 textures

作为 mesh 对象的一个内部变量，等待后续实时绘制。

```
class Model
{
    Mesh processMesh(aiMesh* mesh, const aiScene* scene)
    {
        ...

        vector<Texture> textures;

        // 1. diffuse maps
        vector<Texture> diffuseMaps = loadMaterialTextures(material,
aiTextureType_DIFFUSE, "texture_diffuse");
        textures.insert(textures.end(), diffuseMaps.begin(), diffuseMaps.end());
        // 2. specular maps
        // 3. normal maps
        // 4. height maps
        ...

        return Mesh(vertices, indices, textures);
    }
}
```

此处我们也可以通过用文本编辑器查看“backpack.mtl”文件，来溯源材质的读取，其中设置了光照模式为 2（即漫反射+镜面反射）、漫反射图像的路径为“diffuse.jpg”、镜面反射图像的路径为“specular.jpg”等，具体含义可以查阅 mtl 文档，并于 Assimp 文档中的对应参数结合。（注意示例代码中有时将 `normal` 与 `height` 的命名混淆，但不影响纹理处理，normal 在代码里实际处理的是凹凸的高度图）

<https://paulbourke.net/dataformats/mtl/>

https://documentation.help/assimp/material_8h.html

```
# Material Count: 1
illum 2
map_Kd diffuse.jpg
map_Bump normal.png
map_Ks specular.jpg
```

通过对函数的调用分析，在 OurModel 的 Draw 过程中，是调用 Mesh 类的数组实例化变量 meshes 的 Draw 实现的；mesh 数据由 Model 类中前述的几个函数读取，读取后再传入 Mesh 类中，进而调用 OpenGL 函数绘制。同时，我们可以发现，纹理处理后，对应的名字格式为“texture_diffuse1”、“texture_diffuse2”、“texture_specular1”...

```
class Mesh {
    // render the mesh
    void Draw(Shader& shader)
    {
        // bind appropriate textures
        unsigned int diffuseNr = 1;
        ...

        for (unsigned int i = 0; i < textures.size(); i++)
        {
```

```

        glActiveTexture(GL_TEXTURE0 + i); // active proper texture unit before
binding
        // retrieve texture number (the N in diffuse_textureN)
        string number;
        string name = textures[i].type;
        if (name == "texture_diffuse")
            number = std::to_string(diffuseNr++);
        ...

        // now set the sampler to the correct texture unit
        glUniform1i(glGetUniformLocation(shader.ID, (name + number).c_str()), i);
        // and finally bind the texture
        glBindTexture(GL_TEXTURE_2D, textures[i].id);
    }
    ...

    // draw mesh
    // always good practice to set everything back to defaults once configured.
}
}

```

5. 调整相机位置，观察完整模型

在示例代码中，摄像机通过 Camera 类进行了封装，初始相机位置通过 `Camera camera();` 初始化，我们在“**model_loading.cpp**”中直接修改其位置，使摄像机更靠后。

```
Camera camera(glm::vec3(0.0f, 0.0f, 8.0f));
```

可以注释掉鼠标回调与设置函数，方便截图。

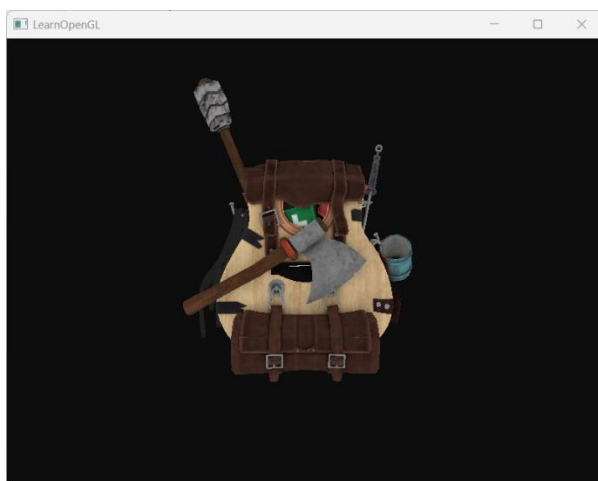
```

// glfwSetCursorPosCallback(window, mouse_callback);
// glfwSetScrollCallback(window, scroll_callback);

// tell GLFW to capture our mouse
// glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);

```

完整模型展示结果如下：



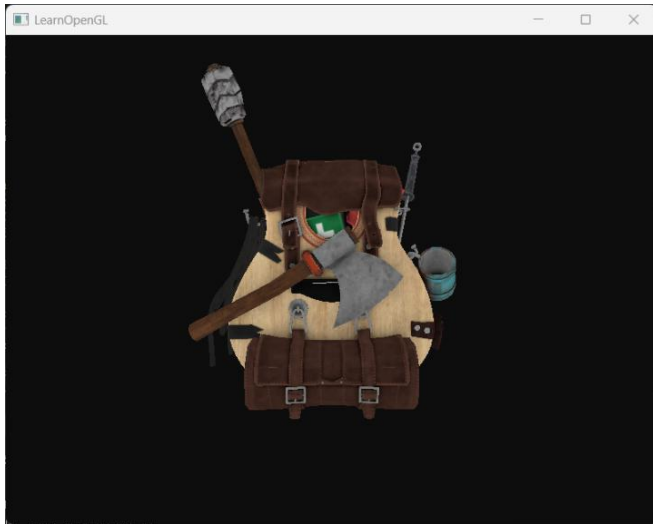
6. 渲染示例材质，得到仅含漫反射、仅含镜面反射的结果。

通过步骤 4 中的分析，我们可以知道，材质的读取在.mtl 文件中设置并由 Assimp 自动读取，材质的处理在 processMesh 函数中，材质的绘制在 Draw 函数中。

processMesh 函数处理得到四种材质的纹理图：漫反射图 `diffuse`（基础颜色）、镜面反射图 `specular`（反射高光）、法线 `normal`（方向）、高度图 `height`（凹凸），纹理分别命名为“`texture_diffuse1`”、“`texture_diffuse2`”、“`texture_specular1`”…。

由于片段着色器的原因，片段着色器每次仅处理一个纹理；我们需要修改片段着色器来实现绘制。

不进行修改，即只包含漫反射的绘制结果：



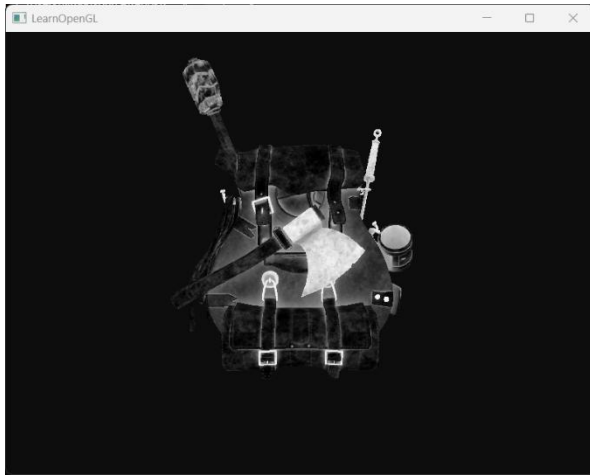
修改片段着色器“`1.model_loading.fs`”，得到只包含镜面反射的绘制结果：

```
#version 330 core
out vec4 FragColor;

in vec2 TexCoords;

uniform sampler2D texture_specular1;

void main()
{
    FragColor = texture(texture_specular1, TexCoords);
}
```



我们继续尝试同时包含漫反射与镜面反射的绘制结果：

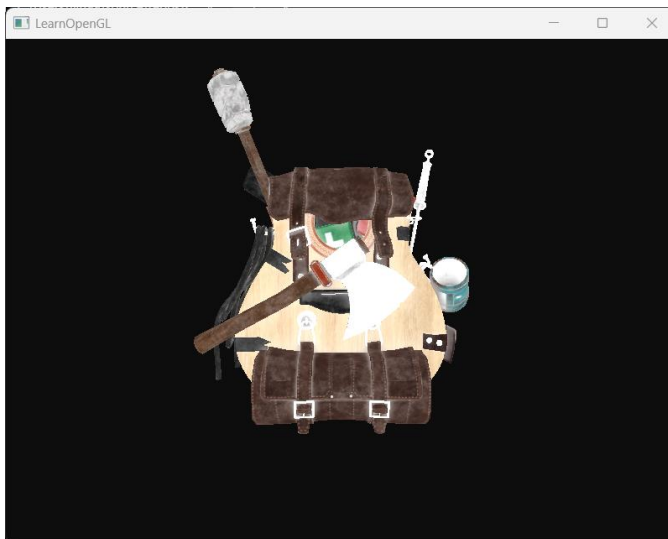
```
#version 330 core
out vec4 FragColor;

in vec2 TexCoords;

uniform sampler2D texture_diffuse1;
uniform sampler2D texture_specular1;

void main()
{
    vec3 diffuse = texture(texture_diffuse1, TexCoords).rgb;
    vec3 specular = texture(texture_specular1, TexCoords).rgb;

    vec3 result = diffuse + specular;
    FragColor = vec4(result, 1.0);
}
```



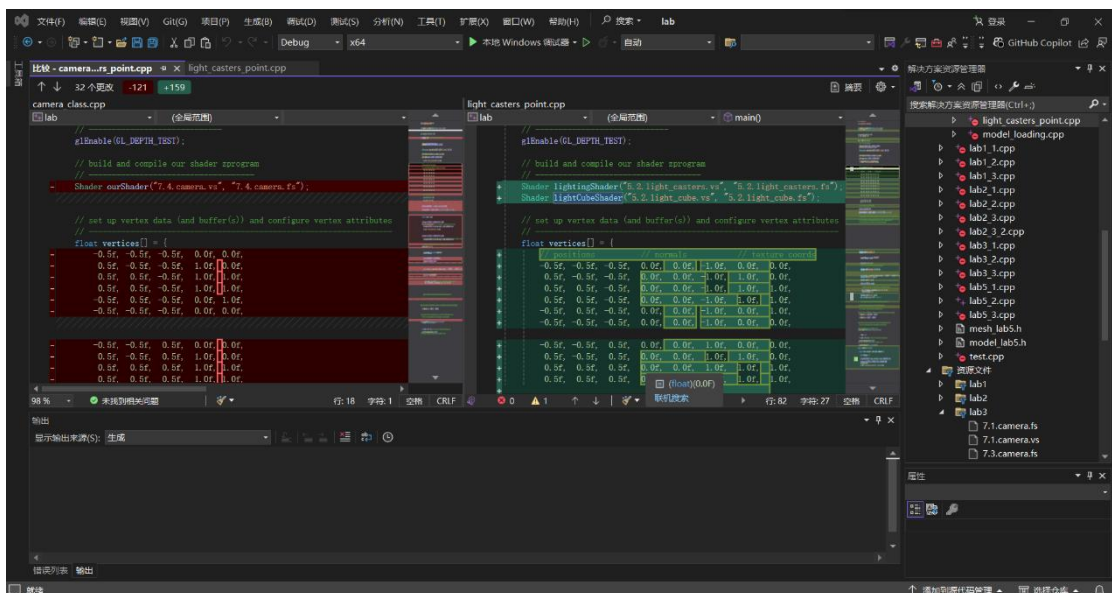
7. 对比“light_casters_point.cpp”与“camera_class.cpp”，结合教程“基础光照”、“材质”与“投光物”三节理解点光源的设置

对比代码可知，在“light_casters_point.cpp”中，**lightingShader 着色器**渲染物体的位置及纹理等，**lightCubeShader 着色器**渲染可视化的光源方块。

材质通过 loadTextures 读取文件路径中的纹理图，得到的纹理索引分别放到变量 diffuseMap、specularMap，再通过 lightingShader.setInt()直接映射到着色器中（由于代码中仅两个纹理，直接按顺序设置为 0、1），并在每次绘制循环前绑定两个纹理。可以注意到“light_casters_point.cpp”中的 loadTextures 函数与“model.h”中的 TextureFromFile 函数完全一致。

“model_loading.cpp”中直接使用默认纹理图渲染，材质及纹理不变化，因此不在着色器中进行纹理处理；而在“light_casters_point.cpp”中，由于点光源可能变化或移动，纹理通过片段着色器“5.2.light_casters.fs”实时更新。

初始的纹理、光照信息通过 lightingShader.setInt()、lightingShader.setVec3()、lightingShader.setFloat() 统一设定，具体参数结合“纹理”、“投光物”两节理解，如 light.ambient、light.diffuse、light.specular 表示光照强度，light.constant、light.linear、light.quadratic 表示光衰减的多次项系数。也需要注意分辨 material.ambient、material.diffuse、material.specular，这里的 specular 等表示物体材质的反射强度。通过读取纹理图得到；之前的 light.specular 表示光源照射的光照强度，是我们固定设置的。



同时，结合片段着色器“5.2.light_casters.fs”中的代码，我们也可以直观理解：ambient 环境光相当于物体底色，其材质颜色设置为与漫反射材质颜色同样的值，再通过 light.ambient 中光源的设置调节底色明暗。

diffuse 漫反射光，由于假设为粗糙表面，材质颜色为物体原色，light.diffuse 与法线结合，共同确定物体的漫反射颜色（即正对光源的位置亮度更高，侧对的亮度低）。

specular 镜面反射光，由于假设为镜面反射，材质颜色为灰度图、表示反射亮度强弱。material.specular 与法线、反光度结合，共同确定物体的镜面反射颜色。

包括环境光在内三种光照的都会随离光源的距离变大而衰减。

```
void main()
{
    // ambient
```

```

vec3 ambient = light.ambient * texture(material.diffuse, TexCoords).rgb;

// diffuse
vec3 norm = normalize(Normal);
vec3 lightDir = normalize(light.position - FragPos);
float diff = max(dot(norm, lightDir), 0.0);
vec3 diffuse = light.diffuse * diff * texture(material.diffuse, TexCoords).rgb;

// specular
vec3 viewDir = normalize(viewPos - FragPos);
vec3 reflectDir = reflect(-lightDir, norm);
float spec = pow(max(dot(viewDir, reflectDir), 0.0), material.shininess);
vec3 specular = light.specular * spec * texture(material.specular, TexCoords).rgb;

// attenuation
float distance = length(light.position - FragPos);
float attenuation = 1.0 / (light.constant + light.linear * distance +
light.quadratic * (distance * distance));

ambient *= attenuation;
diffuse *= attenuation;
specular *= attenuation;

vec3 result = ambient + diffuse + specular;
FragColor = vec4(result, 1.0);
}

```

8. 设置单光源并计算光照，得到仅含漫反射、仅含镜面反射、二者均含的结果。

通过步骤 7 中的分析，我们将任务二中已有的着色器调用代码修改为“light_casters_point.cpp”中调用的 lightingShader、lightCubeShader 两个着色器。注意下述代码所处位置与“light_casters_point.cpp”中代码位置相同，需对比修改。

在任务二修改后的“**model_loading.cpp**”基础上，设置光源位置：

```
glm::vec3 lightPos(1.2f, 1.0f, 2.0f);
```

为便于理解，我们先不可视化光源，而是仅设置 lightingShader 着色器：

```

// build and compile shaders
Shader lightingShader("5.2.light_casters.vs", "5.2.light_casters.fs");

```

结合“model_loading.cpp”读取材质，并设置材质的纹理名；注意此处不需要设置变量 diffuseMap、specularMap，这是因为两张材质图通过 Assimp 读取并通过 model.h 中的 processMesh 函数处理，最后通过 ourModel.Draw 绘制。为了使得读取纹理名与片段着色器中的相对应，我们需要修改“model.h”中的 processMesh 函数、“mesh.h”中的 Draw 函数中的纹理名，或者选择修改片段着色器中的纹理变量名；此处我们选择修改片段着色器“**5.2.light_casters.fs**”：

...

```

struct Material {
    float shininess;
};
uniform sampler2D texture_diffuse1;
uniform sampler2D texture_specular1;
...
void main()
{
    ...

    vec3 ambient = light.ambient * texture(texture_diffuse1, TexCoords).rgb;
    ...

    vec3 diffuse = light.diffuse * diff * texture(texture_diffuse1, TexCoords).rgb;
    ...

    vec3 specular = light.specular * spec * texture(texture_specular1, TexCoords).rgb;
    ...
}

```

在代码“**model_loading.cpp**”中的渲染循环中，将光源属性与物体材质属性传入着色器中：

```

// view/projection transformations
glm::mat4 projection = glm::perspective(glm::radians(camera.Zoom),
(float)SCR_WIDTH / (float)SCR_HEIGHT, 0.1f, 100.0f);
glm::mat4 view = camera.GetViewMatrix();
glm::mat4 model = glm::mat4(1.0f);

model = glm::mat4(1.0f);
model = glm::translate(model, lightPos);
model = glm::scale(model, glm::vec3(0.2f)); // a smaller cube

// be sure to activate shader when setting uniforms/drawing objects
lightingShader.use();
lightingShader.setVec3("light.position", lightPos);
lightingShader.setVec3("viewPos", camera.Position);

// light properties
lightingShader.setVec3("light.ambient", 0.2f, 0.2f, 0.2f);
lightingShader.setVec3("light.diffuse", 0.5f, 0.5f, 0.5f);
lightingShader.setVec3("light.specular", 1.0f, 1.0f, 1.0f);
lightingShader.setFloat("light.constant", 1.0f);
lightingShader.setFloat("light.linear", 0.09f);
lightingShader.setFloat("light.quadratic", 0.032f);

// material properties
lightingShader.setFloat("material.shininess", 32.0f);

```

```

lightingShader.setMat4("projection", projection);
lightingShader.setMat4("view", view);

// render the loaded model
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
lightingShader.setMat4("model", model);

ourModel.Draw(lightingShader);

```

由于光源属性直接传入片段着色器中，我们可以通过修改 `light.diffuse` 和 `light.specular` 来实现仅含漫反射、仅含镜面反射、二者均含的效果。对比时，环境光照 `light.ambient` 设为 0。

我们先来测试与任务二相同的效果，任务二中由于没有光源，相当于光为最大平行光，且不随距离而衰减，不随法线方向而改变，且无环境光（实际上在任务二中与漫反射相同）。我们需要先修改“5.2.light_casters.fs”中的两行代码。

```

float diff = 1.0;
...
float spec = 1.0;

```

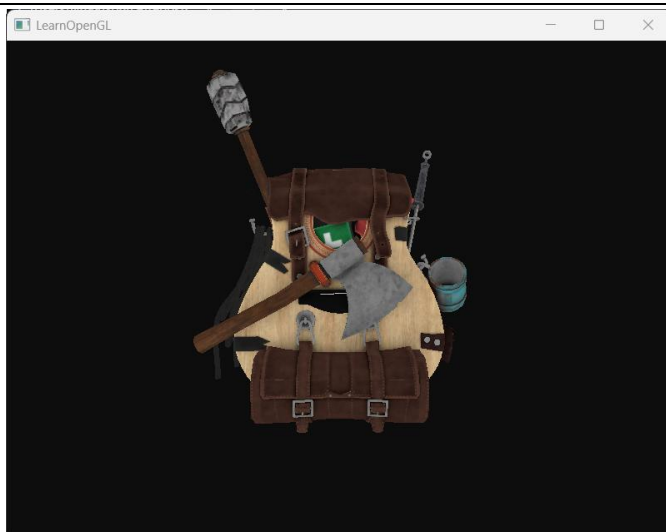
仅含漫反射需要在“model_loading.cpp”设置相应光照：

```

// light properties
lightingShader.setVec3("light.ambient", 0.0f, 0.0f, 0.0f);
lightingShader.setVec3("light.diffuse", 1.0f, 1.0f, 1.0f);
lightingShader.setVec3("light.specular", 0.0f, 0.0f, 0.0f);
lightingShader.setFloat("light.constant", 1.0f);
lightingShader.setFloat("light.linear", 0.0f);
lightingShader.setFloat("light.quadratic", 0.0f);

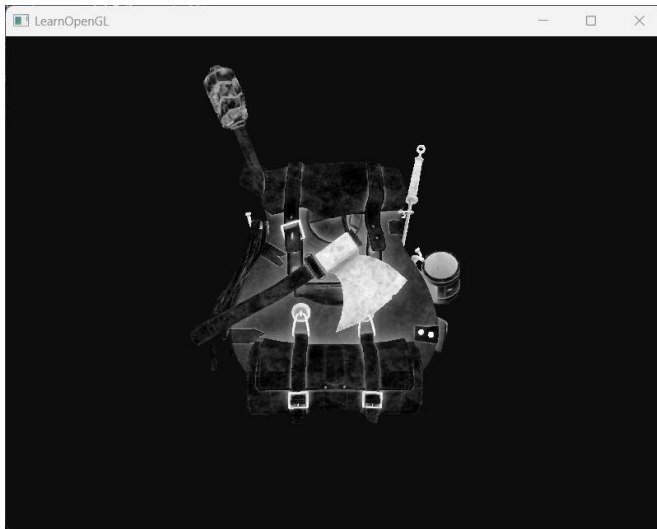
// material properties
lightingShader.setFloat("material.shininess", 100.0f);

```



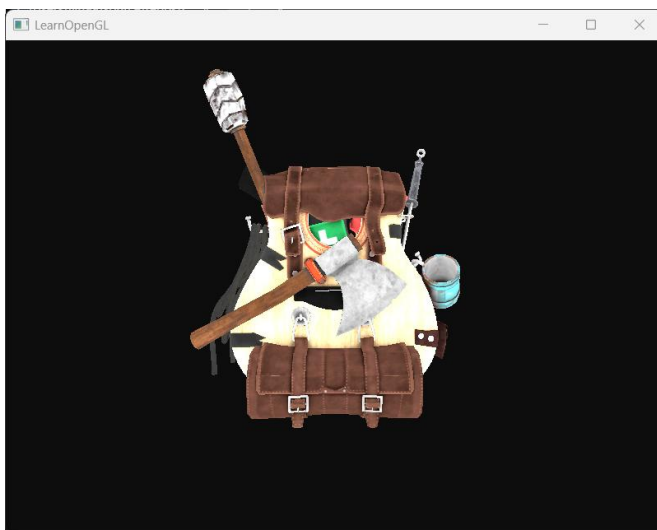
仅含镜面反射需要在仅含漫反射的基础上修改相应光照：

```
// light properties
lightingShader.setVec3("light.diffuse", 0.0f, 0.0f, 0.0f);
lightingShader.setVec3("light.specular", 1.0f, 1.0f, 1.0f);
```



两种均含也需要修改相应光照：

```
// light properties
lightingShader.setVec3("light.diffuse", 1.0f, 1.0f, 1.0f);
lightingShader.setVec3("light.specular", 1.0f, 1.0f, 1.0f);
```



在“**model_loading.cpp**”增加 lightCubeShader 着色器的调用，自定义可视化的光源方块：

```
Shader lightCubeShader("5.2.light_cube.vs", "5.2.light_cube.fs");

// set up vertex data (and buffer(s)) and configure vertex attributes
// -----
float vertices[] = {
    // positions
    -0.5f, -0.5f, -0.5f, // 0
```

```

        -0.5f,  0.5f, -0.5f, // 1
        -0.5f, -0.5f,  0.5f, // 2
        -0.5f,  0.5f,  0.5f, // 3
        0.5f, -0.5f, -0.5f, // 4
        0.5f,  0.5f, -0.5f, // 5
        0.5f, -0.5f,  0.5f, // 6
        0.5f,  0.5f,  0.5f, // 7
    };

    unsigned int indices[] = {
        0, 4, 5,
        5, 1, 0,
        2, 6, 7,
        7, 3, 2,
        3, 1, 0,
        0, 2, 3,
        7, 5, 4,
        4, 6, 7,
        0, 4, 6,
        6, 2, 0,
        1, 5, 7,
        7, 3, 1
    };

    // positions all containers
    glm::vec3 cubePositions[] = {
        glm::vec3(0.0f,  0.0f,  0.0f),
        glm::vec3(2.0f,  5.0f, -15.0f),
        glm::vec3(-1.5f, -2.2f, -2.5f),
        glm::vec3(-3.8f, -2.0f, -12.3f),
        glm::vec3(2.4f, -0.4f, -3.5f),
        glm::vec3(-1.7f,  3.0f, -7.5f),
        glm::vec3(1.3f, -2.0f, -2.5f),
        glm::vec3(1.5f,  2.0f, -2.5f),
        glm::vec3(1.5f,  0.2f, -1.5f),
        glm::vec3(-1.3f,  1.0f, -1.5f)
    };

    unsigned int VBO, lightCubeVAO, EBO;
    glGenVertexArrays(1, &lightCubeVAO);
    glGenBuffers(1, &VBO);
    glGenBuffers(1, &EBO);

    // bind the Vertex Array Object first, then bind and set vertex buffer(s), and then
    configure vertex attributes(s).

```

```

glBindVertexArray(lightCubeVAO);

glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices, GL_STATIC_DRAW);

// position attribute
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);

```

在渲染循环内直接绘制光源方块：

```

// also draw the lamp object
lightCubeShader.use();
lightCubeShader.setMat4("projection", projection);
lightCubeShader.setMat4("view", view);
lightCubeShader.setMat4("model", model);

glBindVertexArray(lightCubeVAO);
glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_INT, 0);
glDrawArrays(GL_TRIANGLES, 0, 36);

```

渲染结束时，删除对应数组，防止内存溢出

```

glDeleteVertexArrays(1, &lightCubeVAO);
glDeleteBuffers(1, &VBO);

```

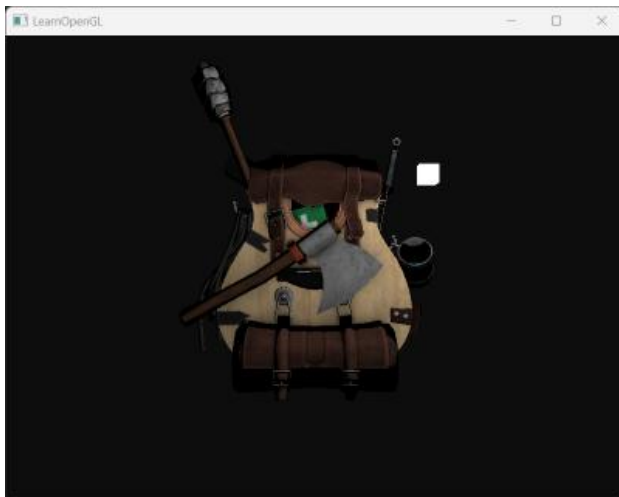
重新将片段着色器“5.2.light_casters.fs”修改为原版，即增加法线与光照距离等信息。

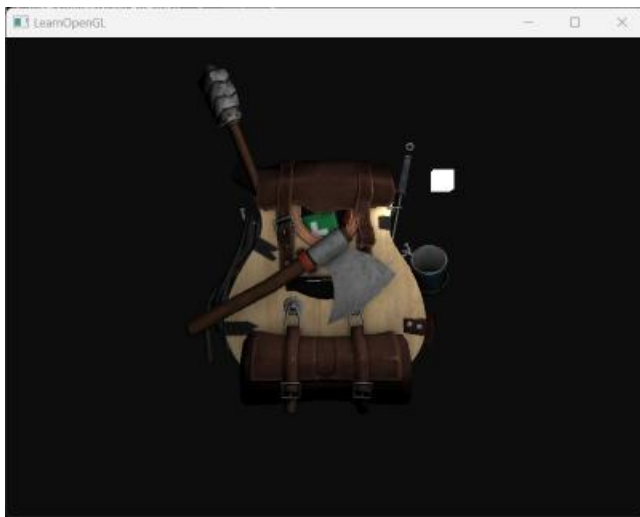
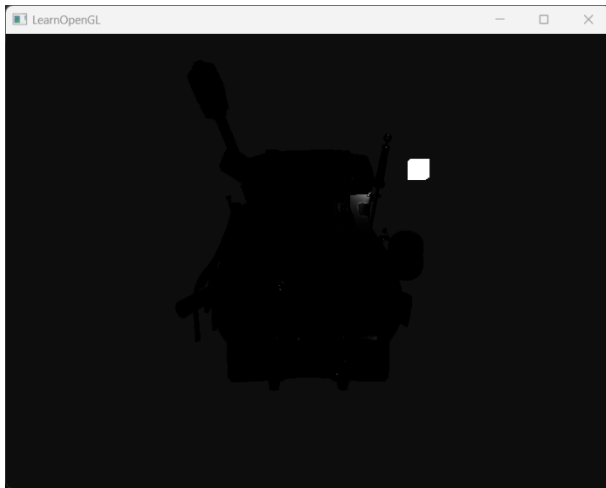
```

float diff = max(dot(norm, lightDir), 0.0);
...
float spec = pow(max(dot(viewDir, reflectDir), 0.0), material.shininess);

```

绘制三种不同材质的结果，得到：





可以直观看得到物体的镜面反射效果。

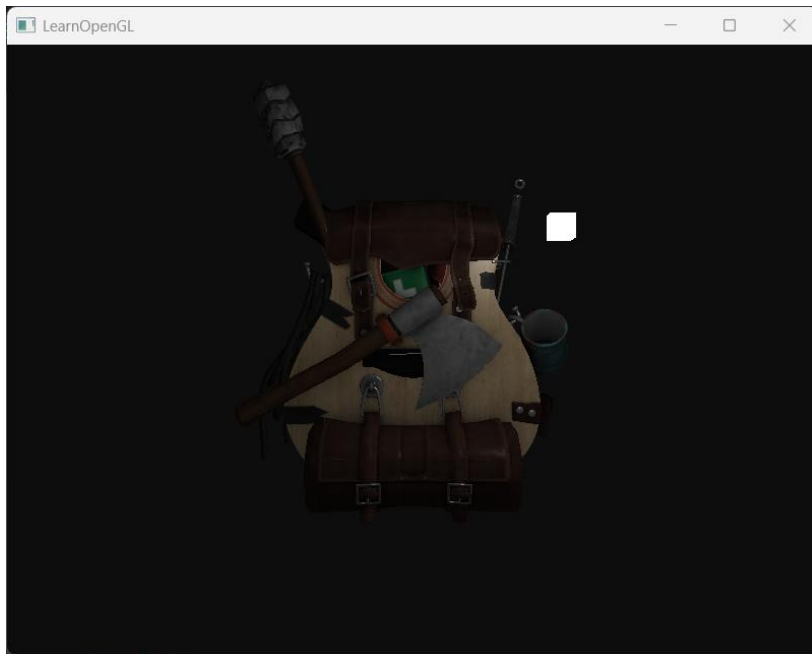
9. 你也可以自定义展示结果。

上面的展示中，为了方便对比，我们固定了相机位置，并设置环境光照为 0、反光度为 100、无衰减等。

我们也可以自由移动视角、设置衰减等来观察光照结果，如使用“light_casters_point.cpp”中的原有光照配置来设置“**model_loading.cpp**”。

```
// light properties
lightingShader.setVec3("light.ambient", 0.2f, 0.2f, 0.2f);
lightingShader.setVec3("light.diffuse", 0.5f, 0.5f, 0.5f);
lightingShader.setVec3("light.specular", 1.0f, 1.0f, 1.0f);
lightingShader.setFloat("light.constant", 1.0f);
lightingShader.setFloat("light.linear", 0.09f);
lightingShader.setFloat("light.quadratic", 0.032f);

// material properties
lightingShader.setFloat("material.shininess", 32.0f);
```

答疑：

1. 我希望直接在项目中查看模型文件，却运行错误。

在将.obj、.png、.mtl 等文件添加到“项目”-“资源文件”-“现有项”之后，还需要**右击.obj 文件，选择“属性”-“配置属性”-“常规”-“从生成中排除”-“是”**，将其从项目生成中排除；这是因为.obj 后缀除了可以表示 mesh 格式的模型文件后缀外，也可以表示 c 语言编译产生的中间代码文件，VS 会误将其视作后者运行。

2. 我尝试按照旧版任务清单修改了“model.h”，确实可以实现任务二“镜面反射”的效果。

这是由于在仅有单个纹理时，该纹理单元默认激活；导致 OpenGL 在处理时，即使纹理名不对应，也能匹配上，并顺利运行。之前布置任务时未进行充分测试，代码实际逻辑是有问题的。实际需要修改着色器来达到可靠结果。